

Your Global Claude Code Skills

A plain-English guide to every skill and helper now available in all your projects

Generated June 22, 2026 · 345 entries total

These are installed at the **global level**, so they're available in every session and every project on your machine. A 'skill' is a focused set of instructions the assistant can pull in automatically when your request matches what the skill is for — you don't have to call it by name. An 'agent' is a specialized helper the assistant can hand a specific job to. You don't need to memorize any of this; the list is just so you can see what's on tap.

Section	What it is	Count
ponytail skills	Tools that fight over-complication and keep things simple	6
ECC skills	A large toolkit covering coding, testing, security, writing, business & more	271
ECC agents	Specialist helpers, each focused on one kind of job	67
Other	Setup helper already on your machine	1

ponytail skills (6)

Keep code simple and avoid doing more than necessary.

ponytail — Pushes for the simplest, laziest solution that still gets the job done. Before any work it asks whether the task even needs doing at all, prefers built-in tools over extra add-ons, and one line of code over fifty. Has three strengths: lite, full, and ultra.

ponytail-review — Looks over a batch of recent code changes hunting only for needless complication — things that could be deleted or swapped for a simpler built-in option. Gives a short list of what to cut and what to replace it with.

ponytail-audit — Same idea as ponytail-review, but scans your entire project instead of just the latest changes. Produces a ranked list of what to delete, simplify, or replace with simpler standard tools.

ponytail-debt — Gathers all the 'I took a shortcut here' notes that ponytail leaves behind in the code into one tidy to-do list, so deferred work gets tracked instead of forgotten.

ponytail-gain — Shows a simple scoreboard of how much ponytail typically saves — less code, lower cost, faster — based on its test results.

ponytail-help — A quick cheat-sheet that lists all the ponytail modes and commands and how to use them.

ECC skills (271)

A broad toolkit. Listed alphabetically. The assistant reaches for one when your task matches it.

accessibility — Helps you build and check websites and apps so people with disabilities (like those who are blind or can't use a mouse) can actually use them, following the official accessibility rulebook. Use it when you want to make sure your product works for everyone.

agent-architecture-audit — Gives an AI assistant a thorough health check, looking through all its moving parts to find problems like it forgetting things, misusing its tools, or getting stuck repeating itself, then tells you exactly how to fix each issue. Use it when you're building an AI helper and want to know what's broken.

agent-eval — Sets up a fair contest between different AI coding assistants on tasks you choose, then reports which one does the job, how much it costs, how long it takes, and how reliable it is. Use it when deciding which AI coding tool to use.

agent-harness-construction — Helps you set up an AI assistant's "controls" — the buttons it can press and the way it sees information — so it succeeds at tasks more often. Use it when an AI helper keeps failing and you want to give it better tools to work with.

agent-introspection-debugging — A step-by-step routine for figuring out why an AI assistant messed up: it records what happened, diagnoses the cause, safely recovers, and writes up a report. Use it when your AI helper fails and you need to understand why.

agent-payment-x402 — Lets an AI assistant pay for things on its own using a digital wallet, with spending limits per task so it can't overspend. Use it when you want an AI helper to handle small purchases or fees automatically and safely.

agent-self-evaluation — After an AI finishes a job, it grades its own work on five things — accuracy, completeness, clarity, usefulness, and brevity — and suggests how to do better, with specific reasons for each score. Use it to get a quality check on the AI's output.

agent-sort — Looks at a specific project and sorts a big bundle of AI tools into "use every day" versus "keep on the shelf for occasional use," so you only load what that project actually needs. Use it to trim a bloated

toolset down to the essentials.

agentic-engineering — A working style for an AI that builds software: it tests its ideas early, breaks big jobs into smaller pieces, and picks the cheapest AI model that can handle each part. Use it as a smart, budget-conscious way to get software built.

agentic-os — Helps you build a long-running team of cooperating AI helpers that remember things using plain files instead of a database, run tasks on a schedule, and keep track of their own state. Use it to create a persistent, multi-assistant setup.

ai-first-engineering — A way of organizing a software team for situations where AI writes most of the actual code, covering how people and AI should divide the work. Use it when your team leans heavily on AI to produce code.

ai-regression-testing — Testing methods for software written with AI's help, including safe testing without touching real data and automatic bug checks, designed to catch mistakes the AI misses because the same AI both wrote and reviewed the code. Use it to catch AI's blind spots.

android-clean-architecture — A tidy way to organize the code of phone apps (for Android phones) so the different parts stay neatly separated and easy to change. Use it when building or restructuring an Android app.

angular-developer — Writes code and gives advice for building websites with Angular, a popular toolkit for web apps, covering things like forms, page navigation, styling, and testing. Use it when building or improving an Angular website.

api-connector-builder — Helps you add a new connection between your software and an outside service, copying the exact style of connections your project already uses so everything stays consistent. Use it when plugging in one more outside service.

api-design — Guidance for designing the "doorways" that let different software programs talk to each other cleanly — covering naming, error messages, handling large lists, and limiting how often someone can call in. Use it when building these connection points for real-world use.

architecture-decision-records — Automatically notices when an important design choice is made while working, then records what was decided, what else was considered, and why — so future developers understand the reasoning. Use it to keep a clear history of big decisions.

article-writing — Writes long-form pieces like blog posts, guides, tutorials, and newsletters in a specific voice copied from examples you provide. Use it when you want polished, on-brand writing longer than a paragraph.

automation-audit-ops — Takes inventory of all your automatic background jobs and connections, showing which are working, broken, duplicated, or missing, before you start fixing anything. Use it to get a clear picture of your automation setup first.

autonomous-agent-harness — Turns the AI coding tool into a self-running assistant that remembers things, runs tasks on a schedule, and works on its own without needing other outside software. Use it when you want continuous, hands-off AI operation.

autonomous-loops — Recipes for setting up AI that runs on its own, from simple one-thing-after-another sequences to more complex setups with several AI helpers working together. Use it when designing self-running AI workflows.

backend-patterns — Best practices for building the behind-the-scenes part of an app — the server, the database, and the connection points — using common web tools. Use it when working on the server side of an app.

benchmark-methodology — Guidance on how to properly measure software performance so your speed tests are fair and trustworthy. Use it when you need to test how fast something runs the right way.

benchmark-optimization-loop — Repeatedly tries many versions of something, measures each one, and picks the fastest or cheapest. Use it when you want to make something run faster and want the computer to test variations for you.

benchmark — Measures how fast your software runs as a baseline, spots when a change made it slower, and compares different tool choices. Use it to track performance and catch slowdowns before and after changes.

blender-motion-state-inspection — Helps inspect 3D characters and their animations in Blender (3D modeling software) — checking poses, whether feet touch the ground, and which way they face — when a simple picture isn't enough to tell. Use it when fine-tuning 3D character animation.

blueprint — (Description not provided in source.)

brand-discovery — (Description not provided in source.)

brand-voice — Studies a person's or company's real writing — posts, essays, documents — to build a profile of their unique style, then reuses that style across new content so it doesn't sound like generic AI. Use it to keep writing consistent and human-sounding.

browser-qa — Automatically opens a web browser and clicks through your website to check that everything looks and works right after you release a new feature. Use it to test the visible parts of a site without doing it by hand.

bun-runtime — Guidance on using Bun, a fast all-in-one tool for running and building web software, including when to pick it over the more common alternative (Node). Use it when deciding on or switching to Bun.

canary-watch — Keeps an eye on a live website right after you publish changes, checking that pages load, files are there, and nothing broke or got slower. Use it as a safety check right after a release.

carrier-relationship-management — Expert know-how for managing trucking and shipping companies you work with — negotiating prices, scoring their performance, dividing up shipments, and keeping good relationships. Use it when handling freight carriers and shipping strategy.

cisco-ios-patterns — Review guidance for the settings and commands on Cisco networking equipment (the gear that runs business networks), helping you check configurations and make changes safely. Use it when reviewing or changing Cisco network setups.

ck — Gives the AI coding tool a memory for each project so it automatically remembers the project's context every time you start, and tracks your work sessions. Use it so the AI doesn't forget what your project is about.

claude-devfleet — Coordinates a team of AI coding helpers working on separate copies of your project at the same time, planning the work, sending out the helpers, and tracking their progress. Use it for tackling big coding jobs in parallel.

click-path-audit — Traces every button and tap in your app through everything that happens when pressed, to find bugs where each step works alone but together they cancel out or leave things broken. Use it when buttons seem broken but normal debugging finds nothing.

clickhouse-io — Best practices for using ClickHouse, a database built for crunching huge amounts of data quickly for reports and analysis. Use it when working with very large datasets for analytics.

code-tour — Creates guided, step-by-step walkthroughs of a codebase that point at real files and lines, like a tour for new team members or to explain how something works. Use it for onboarding or explaining how code fits together.

codebase-onboarding — Studies an unfamiliar pile of code and produces a beginner's guide — a map of how it's built, where to start, and the team's conventions. Use it when you're new to a project and need to get oriented fast.

codehealth-mcp — Continuously checks the quality and structure of your code, before and after changes, and can block low-quality code from being saved. Use it to keep code clean and catch when AI edits made things worse.

coding-standards — Basic shared rules for writing clean code across projects — things like good names and readability. Use it for general code quality, with other skills handling specific tools.

competitive-platform-analysis — (Description not provided in source.)

competitive-report-structure — (Description not provided in source.)

compose-multiplatform-patterns — Patterns for building app screens that work across phones and other devices using a shared toolkit, covering layout, navigation, and appearance. Use it when building cross-device app interfaces with this toolkit.

config-gc — Cleans up the AI coding tool's own settings by finding old, duplicate, or useless items and walking you through deleting them one by one. Use it when your setup has gotten cluttered and bloated.

configure-ecc — A guided installer that walks you through picking and setting up the AI tools and rules you want, making sure everything is in the right place. Use it when first setting up this collection of AI tools.

connections-optimizer — Cleans up and grows your social media network (on X and LinkedIn) — suggesting who to unfollow, who to add, and drafting friendly outreach messages in your own voice. Use it to reorganize your professional and social connections.

content-engine — Builds a system for creating social media content tailored to each platform (X, LinkedIn, TikTok, YouTube, newsletters), including adapting one piece of content cleanly for many platforms. Use it when you want posts, scripts, or a content schedule.

content-hash-cache-pattern — Saves the results of slow file-processing work so it doesn't have to redo it, using a fingerprint of each file's contents so the saved result updates automatically when the file changes. Use it to speed up repeated file processing.

context-budget — Checks how much of the AI's limited "working memory" is being eaten up by its various tools and settings, then points out waste and what to trim. Use it when the AI is loading too much and you want to free up space.

continuous-agent-loop — Patterns for AI that runs on its own continuously, with built-in quality checks and ways to recover when something goes wrong. Use it for reliable, self-running AI workflows.

continuous-learning-v2 — A learning system that quietly watches your work sessions, picks up small lessons with a confidence rating, and grows them into reusable tools, keeping lessons separate per project so they don't mix. Use it to have the AI learn from how you work.

continuous-learning — An older, retired version of the learning system above; you're told to use the newer version instead. It's no longer meant to be used.

cost-aware-llm-pipeline — Methods to spend less money on AI services by sending easy tasks to cheaper AI and only using expensive AI when needed, plus tracking spending and reusing past work. Use it to cut your AI bills.

cost-tracking — Tracks and reports how much you're spending on AI usage, broken down by model, session, or date, and against any budgets. Use it when you want to know where your AI money is going.

council — Brings together four different "voices" to debate a tough decision from different angles before you choose, when there are several reasonable options. Use it for hard tradeoffs and go/no-go calls.

cpp-coding-standards — Rules for writing clean, safe, modern code in the C++ programming language, based on the official community guidelines. Use it when writing or reviewing C++ code.

cpp-testing — Guidance specifically for writing and fixing tests for C++ programs, including setting up testing tools and diagnosing tests that fail unpredictably. Use it only when working on C++ tests.

crosspost — Distributes your content across several social platforms (X, LinkedIn, Threads, Bluesky), reshaping it to fit each one rather than posting the same thing everywhere. Use it to share content widely without it feeling copy-pasted.

csharp-testing — Patterns for writing tests for software built in C# and .NET (a Microsoft programming toolkit), including how to organize and structure the tests well. Use it when testing C# software.

customer-billing-ops — Handles customer billing tasks like subscriptions, refunds, cancellations, and plan changes using connected payment tools like Stripe. Use it when helping a customer with billing or managing payments.

customs-trade-compliance — Expert know-how for shipping goods across borders legally — handling customs paperwork, classifying products for tariffs, reducing import duties, and following each country's rules. Use it when dealing with imports, exports, and customs.

dart-flutter-patterns — Ready-to-use patterns for building phone and web apps with Flutter (a popular app-building toolkit) and the Dart language, covering layout, data handling, and navigation. Use it when building apps with Flutter.

dashboard-builder — Builds monitoring dashboards that actually answer the questions operators care about, turning raw numbers into useful displays instead of pretty-but-useless charts. Use it when you need a practical dashboard from your data.

data-scraper-agent — Builds an automatic AI helper that collects public information (jobs, prices, news, sports, etc.) on a schedule, organizes it, stores it, and improves over time — running for free. Use it when you want to automatically track or gather public data.

data-throughput-accelerator — Makes big data-moving jobs — like loading, copying, or transferring large amounts of data — run much faster while keeping the data correct. Use it when slow data transfers are a bottleneck.

database-migrations — Best practices for safely changing how a database is structured and moving data around, including how to undo changes and make updates without taking the system offline. Use it when changing a database in a live system.

deep-research — Digs through many websites for you, pulls the findings together, and hands back a written summary that links to where each fact came from. Use it when you want a well-sourced answer on a topic rather than a quick guess.

defi-amm-security — A safety checklist for the code behind crypto trading and lending pools, looking for ways money could be stolen or prices tricked. Use it before launching that kind of financial app to catch dangerous mistakes.

deployment-patterns — Best practices for taking a finished app and putting it live for real users smoothly, including ways to undo a bad update quickly. Use it when you're ready to release software to the public.

design-system — Builds or checks the shared rulebook of colors, fonts, and spacing that keeps an app looking consistent everywhere. Use it when you want a unified look or want to review changes that affect how

things appear.

django-celery — Helps a website hand off slow chores (like sending emails or processing files) to run quietly in the background so the page stays fast. Use it when adding tasks that shouldn't make the user wait.

django-patterns — Proven ways to structure and build websites with the Django toolkit so they stay organized and fast as they grow. Use it when designing or improving a Django-based web app.

django-security — Guidance for locking down a Django website against common attacks like fake logins or sneaky data theft. Use it when you want to make sure your site is safe for users.

django-tdd — A method for writing automatic tests for a Django website first, then building the feature to pass them, so bugs get caught early. Use it when you want reliable, well-tested code.

django-verification — A final checklist that runs tests, security scans, and tidiness checks on a Django website before it goes live. Use it as a last safety pass before releasing or sharing your work.

dmux-workflows — Lets you run several artificial-intelligence coding assistants side by side on the same screen and coordinate them. Use it when you want multiple assistants working on a project at once.

docker-patterns — Best practices for packaging an app into self-contained boxes so it runs the same on any computer, plus how to wire several of those boxes together. Use it when setting up a consistent local development setup.

documentation-lookup — Pulls the latest official instructions for a software tool or framework instead of relying on possibly outdated memory. Use it when you need accurate, current setup steps or examples for a named tool.

dotnet-patterns — Recommended ways to write clean, reliable programs using Microsoft's C# language and .NET toolkit. Use it when building software in that ecosystem and you want it done the right way.

dynamic-workflow-mode — Helps design custom, flexible setups so an artificial-intelligence assistant can adapt its approach to each task and reuse what it learns. Use it when planning advanced, self-adjusting assistant workflows.

e2e-testing — Sets up automatic tests that act like a real person clicking through your website to confirm everything works end to end. Use it when you want to catch broken features before users do.

ecc-guide — Walks you through what the ECC toolkit currently offers (its assistants, shortcuts, and settings) by checking the live, up-to-date version first. Use it when you're getting started with ECC or want to know what's available.

ecc-tools-cost-audit — An investigation method for figuring out why the ECC tools are running up unexpected costs, like runaway automated tasks or duplicate work. Use it when bills or usage suddenly spike and you need to find the cause with evidence.

email-ops — Helps sort, draft, and send real email, and proves a message actually landed in your Sent folder. Use it when you want help organizing your inbox or confidently sending and following up on messages.

energy-procurement — Expert know-how for buying electricity and gas for businesses at the best price, managing usage charges, and weighing renewable energy deals. Use it when planning how a company sources and pays for its energy.

enterprise-agent-ops — Best practices for running long-running artificial-intelligence helpers safely, keeping an eye on them, and managing them over time. Use it when an organization relies on assistants that operate continuously.

error-handling — Patterns for dealing gracefully with things that go wrong in software, including retrying, giving up safely, and showing clear messages to users. Use it when you want an app that fails gently instead of crashing or confusing people.

eval-harness — A structured way to grade how well an artificial-intelligence assistant did its job, so you can measure and improve it. Use it when you want hard numbers on an assistant's performance rather than gut feel.

evm-token-decimals — Prevents a subtle money-handling bug in crypto apps where amounts get miscounted because different coins track decimal places differently. Use it when building tools that move or display cryptocurrency balances.

exa-search — A smart web search that understands meaning, used to research topics, find code examples, or look up companies and people. Use it when you need higher-quality search results than a plain keyword search.

fal-ai-media — A single doorway to create images, videos, and audio with artificial intelligence from text or other media. Use it when you want to generate pictures, clips, speech, or sound effects.

fastapi-patterns — Recommended ways to build fast, well-organized web services using the FastAPI toolkit, including security and testing. Use it when creating an application that other programs talk to and you want it done properly.

finance-billing-ops — Helps dig into a company's real sales, pricing, refunds, and billing using the actual records rather than generic advice. Use it when you need an accurate money snapshot or to chase down a billing problem like a double charge.

flox-environments — Sets up a project's tools and dependencies so they work identically on every team member's computer, ending the "it works on my machine" headache. Use it when onboarding people or pinning exact tool versions for a project.

flutter-dart-code-review — A review checklist for apps built with Google's Flutter toolkit, covering quality, speed, accessibility, and good structure. Use it when checking the quality of Flutter app code.

foundation-models-on-device — Guidance for using Apple's built-in artificial intelligence that runs directly on an iPhone (no internet needed) to generate and structure text. Use it when building newer iPhone apps that need on-device smart features.

frontend-a11y — Patterns to make websites usable by people with disabilities, such as those using keyboards only or screen readers. Use it whenever building interactive parts of a site like buttons and forms.

frontend-design-direction — Sets a strong, product-specific visual style for ECC's user interfaces so they look polished and intentional. Use it when building or improving websites, dashboards, or app screens.

frontend-patterns — Proven ways to build the parts of a website people see and interact with, with a focus on speed and good structure. Use it when developing the visible front end of a web app.

frontend-slides — Creates eye-catching, animated web-based presentations from scratch or by converting existing PowerPoint files. Use it when you want to build slides for a talk or pitch and want them to look great.

fsharp-testing — Ways to write automatic tests for programs built in the F# language so they stay reliable. Use it when you want well-tested F# code.

gan-style-harness — A setup where one artificial-intelligence assistant builds something and another critiques it, repeating until the result is high quality, all on its own. Use it when you want assistants to produce polished apps with minimal hand-holding.

gateguard — A safety gate that stops an assistant from editing or running things until it has actually investigated the situation properly. Use it when you want to force careful, fact-checked work instead of rushed guesses.

generating-python-installer — Advanced help turning a Python program into a tiny, fast-starting installer for Windows. Use it only when you need a polished, professional, size-optimized installer, not for simple program packaging.

git-workflow — Best practices for tracking code changes and collaborating with a team, including how to combine everyone's work and resolve clashes. Use it when you want a smooth, organized way for a team to manage code together.

github-ops — Handles the day-to-day running of a project hosted on GitHub, like sorting issues, managing change requests, and releases. Use it when you want help operating a GitHub project beyond basic version control.

golang-patterns — Recommended ways to write clean, efficient programs in the Go language. Use it when building software in Go and you want it done idiomatically.

golang-testing — Methods for writing thorough automatic tests for Go programs, including stress and randomized testing. Use it when you want reliable, well-tested Go code.

google-workspace-ops — Lets you find, edit, summarize, and tidy up Google Drive, Docs, Sheets, and Slides as one smooth workflow. Use it when juggling shared documents, trackers, and slide decks in Google's tools.

healthcare-cdss-patterns — Patterns for building software that helps doctors make safer decisions, like flagging risky drug combinations or checking doses. Use it when developing medical tools that assist clinical judgment.

healthcare-emr-patterns — Patterns for building electronic medical record systems with patient safety and easy data entry in mind. Use it when developing software that stores and handles patients' health records.

healthcare-eval-harness — An automatic safety test suite for medical apps that checks for things like accidental exposure of patient data and blocks unsafe releases. Use it before deploying healthcare software to protect patients.

healthcare-phi-compliance — Patterns for protecting patients' private health and personal information, covering who can see it, tracking access, and scrambling it for safety. Use it when handling sensitive medical data and you must keep it private.

hermes-imports — Turns a person's private local work routines into clean, shareable ECC tools without leaking passwords or personal file paths. Use it when preparing private workflows for safe public reuse.

hexagonal-architecture — A way of structuring software so the core business logic stays separate from outside parts like databases, making it easier to test and change. Use it when you want a clean, swappable, well-organized system design.

hipaa-compliance — Focused guidance for meeting the U.S. healthcare privacy law (HIPAA) when handling patient information. Use it when a task is specifically about following U.S. medical privacy rules.

homelab-network-readiness — A pre-flight checklist to confirm you're ready before changing your home network's security zones, web filtering, or remote access. Use it before tinkering with your router or firewall settings to avoid breaking your network.

homelab-network-setup — Practical advice for planning a home network: routers, switches, Wi-Fi points, address ranges, and avoiding common beginner mistakes. Use it when setting up or upgrading the network in

your home.

homelab-pihole-dns — Help installing and managing Pi-hole, a tool that blocks ads and unwanted sites across your whole home network, plus fixing internet name-lookup problems. Use it when you want network-wide ad blocking or smarter home internet filtering.

homelab-vlan-segmentation — Splits a home network into separate lanes (for smart devices, guests, and trusted gear) so they stay isolated for safety. Use it when you want to keep your IoT gadgets or guests away from your private devices.

homelab-wireguard-vpn — Sets up a secure private tunnel so you can safely reach your home network from your phone or laptop while away. Use it when you want remote access to your home devices over the internet.

hookify-rules — Helps you write the small automatic rules that make a tool react to events on its own. Use it when you want to set up these automatic triggers and need help with the format.

inherit-legacy-style — Teaches an artificial-intelligence coding helper to match the existing style of an older, hand-built project instead of imposing its own habits. Use it when bringing an assistant onto an established codebase so it blends in.

intent-driven-development — Turns a vague or risky feature request into a clear, testable list of what "done" actually means before building it. Use it when a change is fuzzy or high-stakes and you want everyone agreed on the goal first.

inventory-demand-planning — Expert know-how for predicting how much stock a store will need, setting safety buffers, and timing reorders across many locations. Use it when forecasting demand or deciding how much inventory to keep on hand.

investor-materials — Creates and updates fundraising documents like pitch decks, one-pagers, and financial projections, keeping the numbers consistent across all of them. Use it when preparing materials to raise money from investors.

investor-outreach — Drafts the emails you send to potential investors: cold intros, warm introductions, follow-ups, and update notes. Use it when reaching out to investors and you want concise, personalized messages.

ios-icon-gen — Generates ready-to-use app icons for iPhone projects from Apple's built-in symbols or a huge library of free icons. Use it when you need icons for an iPhone app.

iterative-retrieval — A technique for gradually gathering just the right background information an assistant needs, refining it step by step. Use it when an assistant keeps missing context and you want it to zero in on what matters.

ito-basket-compare — Compares groups of prediction-market bets against your own notes, portfolio, or research, and points out gaps, without giving investment advice. Use it when you want a hands-off side-by-side look at prediction-market options.

ito-data-atlas-agent — Helps plan background assistants that research prediction markets and draft options, while keeping a human in charge of final edits. Use it for designing such research helpers, not for placing actual bets.

ito-market-intelligence — Researches prediction-market events, venues, and related news to produce fact-grounded briefings, without giving investment advice. Use it when you want a read-only overview of what's happening in prediction markets.

ito-trade-planner — Builds a planning worksheet for prediction-market bets, laying out the venues, rules, and manual steps, without recommending positions or placing bets. Use it to organize your own thinking before

acting on your own.

java-coding-standards — A set of style rules for writing clean Java programs, automatically tuned to the popular Spring Boot and Quarkus frameworks. Use it when you want consistent, high-quality Java code.

jira-integration — Connects to Jira, the tool teams use to track tasks and bug reports. Use it to pull up a task, read what needs doing, leave a comment, or mark something as done.

jpa-patterns — Best ways to set up how a Java app stores and reads its information from a database, so data is saved correctly and pages load quickly even with lots of records.

knowledge-ops — Helps you save, organize, and search your collection of notes and documents that may be scattered across different places, and clears out duplicates so you can actually find things later.

kotlin-coroutines-flows — Techniques for letting a phone app (written in the Kotlin language) do several things at once, like loading data while staying responsive, without freezing up or crashing.

kotlin-exposed-patterns — Recommended ways for a Kotlin program to talk to its database, covering how to read and write records safely and keep the connection efficient.

kotlin-ktor-patterns — Patterns for building the behind-the-scenes part of a web service in Kotlin, including handling incoming requests, checking who's logged in, and real-time messaging.

kotlin-patterns — The clean, accepted way to write Kotlin code so it's reliable and easy for other people to read and maintain.

kotlin-testing — Ways to automatically check that Kotlin code works as intended, so bugs get caught before users ever see them.

kubernetes-patterns — Best practices for running apps across many servers using Kubernetes, a system that automatically keeps your software running, restarts it if it crashes, and scales it up when busy.

laravel-patterns — Recommended ways to structure a web app built with Laravel (a popular toolkit for PHP websites), covering how pages, data, and background jobs fit together.

laravel-plugin-discovery — Helps you find and judge add-on packages for Laravel apps, checking whether they're well-maintained and compatible before you rely on them.

laravel-security — Practices for keeping a Laravel website safe, like making sure only the right people can access things and blocking common ways attackers try to break in.

laravel-tdd — Methods for automatically testing a Laravel website to confirm features and logins work correctly before going live.

laravel-verification — A checklist run for Laravel projects that confirms the code is clean, tested, secure, and actually ready to release.

latency-critical-systems — For software that must respond almost instantly, like live dashboards or financial trading systems, where even tiny delays matter. Helps keep things fast and up-to-date.

lead-intelligence — An automated sales research assistant that finds promising potential customers, scores how good a fit they are, figures out who could introduce you, and drafts personalized outreach messages for email and social media.

liquid-glass-design — Apple's glass-like visual style for iPhone apps, where elements look translucent and blurred and react smoothly when you touch them. Use when designing that polished, modern Apple look.

llm-trading-agent-security — Safety measures for AI programs that can move money or make trades on their own, like spending limits and automatic shut-offs, so a mistake or attack can't drain funds.

logistics-exception-management — Hard-won know-how for handling shipping problems such as delays, damaged goods, lost freight, and disputes with carriers, including how and when to escalate and file claims.

make-interfaces-feel-better — Small but meaningful tweaks to an app's look and feel, like spacing, fonts, shadows, and how buttons react, that make a screen feel polished rather than rough.

manim-video — Creates clean animated explainer videos for technical ideas, charts, and diagrams, instead of a person just talking on camera. Use when you want to show a concept visually.

market-research — Investigates a market, competitors, or industry and produces a clear summary with sources, aimed at helping you make business decisions like sizing an opportunity or comparing rivals.

marketing-campaign — Plans and writes a full marketing push from start to finish, including audience research, web page copy, emails, social posts, ads, and a posting schedule, all in one coordinated effort.

mcp-server-patterns — Guidance for building small connector programs that let AI assistants safely use outside tools and data. Use when creating one of these add-ons.

messages-ops — A careful workflow for reading your texts and direct messages, including recovering a recent one-time login code, while keeping clear proof of exactly which inbox was checked.

ml-adoption-playbook — A step-by-step guide for adding smart, learning-from-data features to existing software that didn't have any before, covering how to frame the problem and start simple.

mle-workflow — A professional process for building artificial-intelligence models that learn from data and run reliably in real products, including testing, monitoring, and undoing a bad update.

motion-advanced — Advanced techniques for animations on websites, like drag-and-drop, gestures, and animated text or drawings. Builds on the basic motion toolkit.

motion-foundations — The groundwork for website animations, setting consistent timing and smooth movement while keeping things fast, accessible, and working for everyone. Other animation tools build on this.

motion-patterns — Ready-made animation recipes for common website elements like buttons, pop-up windows, notifications, and page changes.

motion-ui — A complete set of animation tools for websites, used whenever you want to add smooth movement and transitions to a web app.

mysql-patterns — Recommended ways to set up and use a MySQL database (a common way to store an app's data) so it runs fast and reliably under real-world load.

nanoclave-repl — Runs and extends NanoClaw, an in-house lightweight command-line tool for chatting with an AI assistant while it remembers the current session.

nestjs-patterns — Recommended ways to organize the behind-the-scenes part of a web app built with NestJS, keeping the code tidy and checking incoming data is valid.

netmiko-ssh-automation — Safe ways to use a Python tool to remotely log into network equipment like routers and switches, gather information, and make changes carefully without breaking things.

network-bgp-diagnostics — Troubleshooting (look-only, no changes) for the system that decides how internet traffic routes between networks, helping pinpoint why connections between networks aren't working.

network-config-validation — Checks router and switch settings before they go live to catch dangerous commands, duplicate addresses, and other mistakes that could take down a network.

network-interface-health — Diagnoses problems with network connection ports, such as errors, dropped data, and mismatched settings, on routers, switches, and computers.

nextjs-turbopack — Guidance on a faster build tool for websites made with Next.js, helping developers see their changes more quickly and decide which build tool to use.

nodejs-keccak256 — Prevents a sneaky bug in cryptocurrency programming where two similar-looking scrambling methods get mixed up, which would silently break Ethereum-related code.

nutrient-document-processing — Works with documents to convert, scan, extract text, black out sensitive parts, sign, and fill in forms, handling PDFs, Word files, spreadsheets, and images.

nuxt4-patterns — Recommended ways to build fast, reliable websites with Nuxt 4 (a website-building framework), especially loading data smoothly without glitches.

openclaw-persona-forge — Creates a complete "lobster soul" character for the OpenClaw AI agent, generating its personality, backstory, name, rules, and avatar image prompt, optionally based on your preferences or chosen at random.

opensource-pipeline — Takes a private project and prepares it for safe public release by copying it, stripping out secrets and sensitive bits, and packaging it up for sharing.

orch-add-feature — Manages building a brand-new feature from scratch end to end, walking through research, planning, writing and testing the code, review, and a final approval step.

orch-build-mvp — Takes a design or spec document and turns it into a basic working version of a product, building it up in small complete slices until it runs.

orch-change-feature — Manages changing how an existing, working feature behaves, updating both its tests and its code to match the new desired behavior, with review and approval.

orch-fix-defect — Manages fixing a bug by first writing a test that reproduces it, then fixing the code until that test passes, followed by review and approval.

orch-pipeline — The shared engine behind the other "orchestrate" tools, defining the standard step-by-step build process and the points where a human signs off. Rarely used directly.

orch-refine-code — Manages cleaning up and reorganizing existing code to make it tidier without changing what it actually does, confirming the tests still pass throughout.

parallel-execution-optimizer — Helps get a job done much faster by splitting it into independent pieces that run at the same time, without letting the results get sloppy.

perl-patterns — The modern, accepted way to write code in the Perl programming language so it stays robust and easy to maintain.

perl-security — Comprehensive ways to keep Perl programs safe, covering checking incoming data and blocking common attacks on websites and databases.

perl-testing — Ways to automatically check that Perl code works correctly, including measuring how much of the code the tests actually cover.

plan-orchestrate — Reads a multi-step plan and turns each step into ready-to-use instructions for the automated build process, so you don't have to wire it all together by hand. It only writes the instructions; it doesn't run them.

plankton-code-quality — Automatically tidies up and checks your code the moment you save a file, fixing formatting and small issues with AI help as you go.

postgres-patterns — Recommended ways to set up and use a PostgreSQL database (a common way to store app data) for speed, good structure, and security.

prediction-market-oracle-research — Researches betting-style markets where prices reflect the crowd's odds of future events, and explains how to use those odds as a data signal for products or decisions, without giving investment advice.

prediction-market-risk-review — Reviews betting-market and trading workflows for safety, legal, privacy, and data-quality problems before they handle account logins, personal financial data, or real trades.

prisma-patterns — Recommended ways for a TypeScript app to work with its database using Prisma, with warnings about common surprises that can quietly lose or mangle data.

product-capability — Turns a product idea or feature request into a clear implementation plan that surfaces the tricky constraints and open questions before the engineering work begins.

product-lens — Helps check whether a product idea is actually worth building and pressure-tests the direction before anyone commits to building it.

production-audit — Checks whether a finished app is truly ready to go live, looking for what might break in real-world use, all done locally without sending your code to an outside service.

production-scheduling — Expert know-how for planning factory production, like sequencing jobs, balancing the assembly line, reducing downtime between product switches, and handling unexpected disruptions.

project-flow-ops — Coordinates work between GitHub and Linear (two project-tracking tools), sorting through tasks and code-change requests while keeping GitHub public and Linear as the private to-do system.

prompt-optimizer — Improves the instructions you give an AI so it understands you better and gives stronger results.

python-patterns — The clean, accepted way to write Python code so it's reliable and easy for others to read and maintain.

python-testing — Ways to automatically check that Python code works correctly, including writing tests first and measuring how much of the code is covered.

pytorch-patterns — Helps you build and train artificial intelligence models the right way, so the training process is reliable and gives the same results when you run it again. Use it when you're teaching a computer to recognize patterns, like images or text.

quality-nonconformance — Guides factories through what to do when a product fails to meet quality standards: figuring out why it went wrong, fixing the cause, and dealing with bad parts from suppliers. Use it in regulated industries like medical, car, or aerospace manufacturing.

quarkus-patterns — Provides proven ways to build the behind-the-scenes engine of a Java web application that handles messages and requests between systems. Use it when building the server side of an app with the Quarkus toolkit.

quarkus-security — Best practices for keeping a Quarkus-based app safe, covering things like checking who a user is, controlling what they're allowed to do, and protecting passwords and secret keys. Use it when you want to lock down a Quarkus app against attackers.

quarkus-tdd — Helps you write automated checks for a Quarkus app first, then build the feature to pass those checks, so bugs get caught early. Use it when adding features or fixing problems and you want confidence the code works.

quarkus-verification — A final checklist that runs the build, scans for problems and security holes, runs the tests, and reviews changes before you release a Quarkus app. Use it as a last gate before shipping or sharing code.

ralphinho-rfc-pipeline — Coordinates several automated helpers to carry out a written project proposal step by step, with quality checkpoints and an orderly way to combine everyone's work. Use it for managing complex projects broken into many tasks.

react-patterns — Provides good habits for building the visible parts of a website (buttons, forms, pages) using the popular React toolkit, including how to handle data and make pages usable for everyone. Use it when creating or reviewing website interface pieces.

react-performance — A large collection of tips for making React-based websites load and respond faster, organized by what matters most. Use it when a site feels slow or you want to speed it up.

react-testing — Helps you write automated checks that confirm the parts of your website work and stay accessible, and tells you when a simple test is enough versus a full browser test. Use it when testing website interface pieces.

recsys-pipeline-architect — Helps you design systems that pick the best handful of items to show a person, like a social media feed or search results, using a clear six-step recipe. Use it whenever you're building anything that ranks and chooses "top picks" for someone.

recursive-decision-ledger — Runs many trial-and-error attempts at a problem while keeping a visible record of every choice and the reasoning behind it. Use it for tough search or optimization problems where you want to see how the answer was reached.

redis-patterns — Shows proven ways to use Redis, a super-fast temporary data store, for speeding things up, limiting how often something can run, and coordinating between systems. Use it when building apps that need quick access to frequently used data.

regex-vs-llm-structured-text — Helps you decide whether to pull information out of text using simple pattern-matching rules or a smarter artificial intelligence model, recommending the cheaper rule-based approach first. Use it when you need to extract data from text and aren't sure which method to pick.

remotion-video-creation — Best practices for making videos by writing code (instead of using a video editor), covering animations, audio, captions, and more. Use it when creating videos programmatically with the Remotion tool.

repo-scan — Inspects every file in a software project, sorts them by type, spots copied-in outside code, and gives you a clear report on what each part is. Use it when you want a full inventory and health check of a codebase.

research-ops — A workflow for gathering current, verified facts from public sources to answer a question, compare options, or make a recommendation. Use it when you need fresh, evidence-backed information rather than guesswork.

returns-reverse-logistics — Guides businesses through handling product returns: approving them, inspecting items, deciding what to do with them, issuing refunds, spotting fraud, and managing warranty claims. Use it when running the returns side of a retail or product operation.

rules-distill — Reads through a collection of skill guides, pulls out the common principles they share, and turns those into clean reusable rules. Use it when you want to find and write down the shared wisdom across many guides.

rust-patterns — Shows the proper, natural way to write programs in the Rust language so they're safe and fast, covering its trickier concepts like memory ownership. Use it when writing or improving Rust code.

rust-testing — Covers how to write automated checks for Rust programs, including different testing styles and measuring how much of your code is tested. Use it when you want confidence that your Rust code works correctly.

safety-guard — Acts as a safety brake that stops dangerous actions, like deleting important things, especially when working on live systems or letting automated helpers run on their own. Use it to avoid costly accidents.

santa-method — Has two independent reviewers double-check work, and nothing is approved until both agree it's good. Use it when you want extra-strong quality assurance before finalizing output.

pubmed-database — Searches medical and scientific research databases to find studies, articles, and references on health and biology topics. Use it when you need trustworthy medical literature or want to track new research.

uspto-database — Looks up official United States patent and trademark records, including ownership and status. Use it when researching inventions, brand names, or intellectual property.

gget — A quick tool for looking up genetic and biology data, searching gene sequences, and recording your findings. Use it for fast genomics lookups in biology research.

literature-review — A step-by-step method for thoroughly surveying published research on a topic, from planning the search to checking sources and summarizing findings. Use it when you need a careful, well-documented review of what's already known.

scholar-evaluation — Gives structured feedback on academic work like research papers and proposals, judging the quality of evidence, methods, and citations. Use it when reviewing or improving scholarly writing.

search-first — Encourages looking for existing tools and solutions before writing your own code from scratch. Use it whenever you're about to build something that may already exist.

security-bounty-hunter — Searches software projects for serious, real security flaws that could be exploited from the outside and might earn a reward when reported. Use it when hunting for legitimate, reportable vulnerabilities.

security-review — Provides a security checklist and safe patterns for sensitive work like logins, handling user-entered data, secret keys, and payments. Use it whenever you're building features where a security mistake could be costly.

security-scan — Checks your Claude Code setup files for security weaknesses and risky settings. Use it to make sure your own configuration isn't leaving a door open for attacks.

seo — Reviews and improves a website so it shows up better in search engines like Google, covering page speed, content, and behind-the-scenes tags. Use it when you want more people to find your site through search.

skill-comply — Tests whether automated helpers actually follow their instructions by running them through made-up scenarios and reporting how often they obey. Use it when you want to confirm your rules and guides are really being respected.

skill-scout — Searches existing collections, both local and online, to see if a ready-made skill already exists before you build a new one. Use it when you want to create or find a skill for a task.

skill-stocktake — Reviews your collection of skills and commands for quality, either checking just the recently changed ones or doing a full sweep. Use it when auditing and tidying up your set of skills.

social-graph-ranker — Maps out who knows whom across networks like X and LinkedIn and scores the best paths for a warm introduction. Use it when you want the underlying tool for finding connection routes between people.

social-publisher — Schedules and posts social media content across many platforms at once, uploads media, and tracks whether posts went out. Use it when managing social media posting across sites like X, LinkedIn, Instagram, and more.

springboot-patterns — Provides proven ways to build the server side of a Java web application using the Spring Boot toolkit, covering things like handling requests and storing data. Use it for back-end Java work with Spring Boot.

springboot-security — Best practices for keeping a Spring Boot app safe, including verifying users, controlling access, protecting secrets, and limiting abuse. Use it when securing a Spring Boot application.

springboot-tdd — Helps you write automated checks first and then build Spring Boot features to pass them, catching bugs early. Use it when adding features or fixing bugs in a Spring Boot app.

springboot-verification — A final checklist that builds, tests, scans for security issues, and reviews changes before releasing a Spring Boot app. Use it as a last gate before shipping.

strategic-compact — Suggests trimming the conversation's memory at sensible moments so important context isn't lost during long tasks. Use it to keep an assistant on track through a multi-step job.

swift-actor-persistence — Helps Apple-platform apps safely save and recall data without errors from multiple things happening at once, by combining quick memory access with file storage. Use it when building reliable data storage in Swift apps.

swift-concurrency-6-2 — Guides developers in safely doing several things at once in the Swift language, with simpler default behavior and clear ways to push work into the background. Use it when writing Swift code that juggles multiple tasks.

swift-protocol-di-testing — Makes Swift code easier to test by swapping in fake versions of things like files, networks, and outside services. Use it when you want testable, well-structured Swift code.

swiftui-patterns — Shows good ways to build app screens for iPhone and Mac using Apple's SwiftUI toolkit, covering layout, navigation, and performance. Use it when designing user interfaces for Apple devices.

taste — Adds artistic direction to music videos and short clips in a dreamy, hyper-stylized aesthetic, combining mood, color, and beat-matched editing into one production flow. Use it when you want a video to feel intentional and visually coherent, not just functional.

tdd-workflow — Enforces writing automated checks before building features and aims for high test coverage across all levels. Use it when writing new features, fixing bugs, or reshaping code and you want strong test discipline.

team-agent-orchestration — Coordinates groups of automated helpers like a project team, tracking who owns what task and controlling when their work gets combined. Use it for managing squads of agents working together.

team-builder — Lets you interactively pick and launch several automated helpers to work in parallel. Use it when you want to assemble a team of agents for a job.

terminal-ops — A careful workflow for running commands, checking projects, debugging failures, and making small fixes, while keeping exact proof of what was done. Use it when you need commands run with a clear record of the results.

tinystuct-patterns — Expert guidance for building apps with the tinystuct Java framework, covering its features like web routes, the built-in server, and database storage. Use it when working on a tinystuct project.

token-budget-advisor — Advises on managing how much text an artificial intelligence model processes to stay within limits and control costs. Use it when you need to keep an assistant's workload and expense in check.

ui-demo — Records polished demo videos of a website by automatically clicking through it, with a visible cursor and smooth, natural pacing. Use it when you want a professional walkthrough or tutorial video of a web

app.

ui-to-vue — Turns design pictures or screenshots into working website code using the Vue toolkit and popular component kits, in batches. Use it when you have designs to convert into real Vue interface pieces.

uncloud — Manages a small cluster of servers, handling deploying apps, routing web traffic, scaling up, checking logs, and managing storage. Use it when running services on an Uncloud setup.

unified-notifications-ops — Brings all your alerts from different tools into one organized flow, removing duplicates and making sure important ones reach you. Use it when notifications are scattered, repetitive, or overwhelming.

verification-loop — A thorough system for double-checking that work done in a Claude Code session is correct and complete. Use it when you want built-in verification throughout a coding session.

video-editing — Covers the full process of editing real video footage with the help of artificial intelligence, from raw clips to a polished final cut. Use it when you want to edit video, trim footage, or make vlogs and video content.

videodb — A toolkit for working with video and audio: bringing it in from files or live feeds, searching for specific moments, converting formats, adding subtitles and overlays, and setting up live alerts. Use it whenever you need to find, edit, or react to content inside video and audio.

visa-doc-translate — Translates visa paperwork from photos into English and creates a side-by-side document showing both the original and the translation. Use it when preparing visa application documents in another language.

vite-patterns — Shows good ways to set up and use Vite, a tool that quickly bundles and serves website code during development. Use it when working on a project that uses Vite.

vue-patterns — Provides good habits for building websites with the Vue toolkit, covering structure, data handling, and navigation. Use it when working on a Vue or Nuxt project.

windows-desktop-e2e — Automatically tests Windows desktop programs by simulating a real user clicking through them. Use it when you want to verify a Windows app works from start to finish.

workspace-surface-audit — Looks over your whole working setup (the project, connected tools, and configuration) and recommends the most useful skills and helpers for it. Use it when setting up Claude Code or figuring out what your environment can do.

x-api — Connects to X (formerly Twitter) so you can post tweets and threads, read timelines, search, and pull statistics through code. Use it when you want to interact with X automatically rather than by hand.

ECC agents (67)

Specialist helps the assistant can delegate a focused job to.

a11y-architect — Helps make apps and websites usable by everyone, including people with disabilities such as blindness or limited movement. Use it when building screens or menus and you want to be sure they work with screen readers and other assistive tools.

agent-evaluator — Grades the work another helper just produced, scoring it on whether it's correct, complete, clear, useful, and to-the-point. Use it when you want an honest report card on a piece of work plus suggestions to improve it.

architect — Acts like a big-picture planner for software, helping decide how the major pieces should fit together so the system can grow without falling apart. Use it when starting something new or reshaping something large and you need to make smart high-level decisions.

build-error-resolver — Fixes the errors that stop a program from being assembled and run, making the smallest possible changes to get it working again. Use it when the project won't compile and you just want it back up fast.

chief-of-staff — A personal assistant for your messages across email, Slack, LINE, and Messenger that sorts them by importance, drafts replies for you, and nudges you to follow up. Use it when you're drowning in messages across many apps and want help keeping on top of them.

code-architect — Studies how an existing project is already built, then hands you a concrete blueprint for adding a new feature, including which files to touch and in what order. Use it when you want a new feature to fit naturally with what's already there.

code-explorer — Digs through an existing project to map out how a feature actually works, tracing what calls what and how the parts connect. Use it when you need to truly understand a piece of code before changing or building on it.

code-reviewer — A careful second set of eyes that checks freshly written code for mistakes, security holes, and things that will be hard to maintain. Use it right after writing or changing code, before it goes live.

code-simplifier — Cleans up and tidies code so it's easier to read and maintain, without changing what it actually does. Use it when code works but has gotten messy or inconsistent.

comment-analyzer — Checks the notes left inside code (written for humans, not the computer) to see if they're accurate, helpful, and not outdated. Use it when you suspect the explanatory notes no longer match what the code really does.

conversation-analyzer — Reads through a record of a past work session to spot repeated mistakes worth setting up automatic guardrails against. Use it when you want to learn from previous sessions and prevent the same problems from happening again.

cpp-build-resolver — Fixes the errors that stop programs written in the C++ language from being assembled, with the smallest possible changes. Use it when a C++ project won't compile and you want it working again quickly.

cpp-reviewer — An expert reviewer for code written in the C++ language, focused on memory safety, modern style, and speed. Use it whenever you change C++ code and want it checked over.

csharp-reviewer — An expert reviewer for code written in the C# language, watching for security issues, proper style, and performance. Use it whenever you change C# code and want it checked over.

dart-build-resolver — Fixes the errors that stop apps built with Dart and Flutter (a toolkit for phone and desktop apps) from being assembled, with small surgical changes. Use it when a Dart or Flutter project won't build.

database-reviewer — A specialist for databases (the organized stores where apps keep their information) that checks how data is structured and retrieved for speed, safety, and good design. Use it when writing database queries or designing how data will be stored.

django-build-resolver — Fixes the setup and startup errors for projects built with Django (a popular Python toolkit for websites), making minimal changes. Use it when a Django project won't start or install properly.

django-reviewer — An expert reviewer for websites built with Django, checking for security gaps, safe data handling, and solid production-ready practices. Use it whenever you change Django code.

doc-updater — Keeps a project's written guides and maps of how the code is organized up to date as things change. Use it when documentation has fallen behind the actual code.

docs-lookup — Looks up the current, official instructions for a software library or tool and answers your how-do-I questions with real examples. Use it when you need accurate, up-to-date guidance on using some piece of software.

e2e-runner — Tests an app the way a real person would, clicking through whole flows from start to finish to make sure they work, and saves screenshots and recordings of what happened. Use it when you want to confirm key user journeys, like signing up or checking out, actually work.

fastapi-reviewer — Reviews web services built with FastAPI (a Python toolkit for building app backends) for correctness, security, and readiness for real use. Use it when you change FastAPI code and want it checked over.

flutter-reviewer — Reviews apps built with Flutter and Dart (a toolkit for phone and desktop apps) for good building practices, performance traps, and accessibility. Use it whenever you change Flutter code.

fsharp-reviewer — An expert reviewer for code written in the F# language, focused on its particular style and keeping the code safe and correct. Use it whenever you change F# code.

gan-evaluator — Tests a running app, scores it against a checklist of goals, and gives specific feedback on what to fix. Use it as the judge half of a build-test-improve loop that keeps refining an app until it's good enough.

gan-generator — Builds features to match a plan, reads the feedback it gets, and keeps improving until the work meets the quality bar. Use it as the builder half of a loop that repeatedly writes and refines an app.

gan-planner — Turns a one-line idea into a full plan, spelling out the features, the order to build them, the design direction, and how success will be judged. Use it at the very start when you want to expand a rough idea into a complete blueprint.

go-build-resolver — Fixes the errors that stop programs written in the Go language from being assembled, with minimal changes. Use it when a Go project won't compile.

go-reviewer — An expert reviewer for code written in the Go language, focused on its idioms, handling many tasks at once, and performance. Use it whenever you change Go code.

harmonyos-app-resolver — An expert for building apps for HarmonyOS (Huawei's operating system), checking the code follows the platform's recommended patterns and runs well. Use it when working on HarmonyOS or OpenHarmony apps.

harness-optimizer — Tunes the setup that runs your AI helpers so it's more reliable, cheaper, and faster. Use it when you want to get more out of your local agent system.

healthcare-reviewer — Reviews code for medical and health apps with special attention to patient safety, privacy of health records, and the accuracy of medical information. Use it when working on software that handles patient data or clinical decisions.

homelab-architect — Designs a home or small-office network plan based on your equipment, goals, and skill level, laying out safe step-by-step changes you can undo if needed. Use it when setting up or improving a network at home or in a small lab.

java-build-resolver — Fixes the errors that stop programs written in the Java language from being assembled, automatically recognizing common Java setups and applying the right fixes. Use it when a Java project won't build.

java-reviewer — An expert reviewer for code written in the Java language, covering structure, data handling, security, and doing many things at once. Use it whenever you change Java code.

kotlin-build-resolver — Fixes the errors that stop programs written in the Kotlin language from being assembled, with minimal changes. Use it when a Kotlin project won't build.

kotlin-reviewer — A reviewer for code written in the Kotlin language, often used for Android phone apps, checking for good style, safety, and common pitfalls. Use it whenever you change Kotlin code.

loop-operator — Runs and watches automatic AI work loops, stepping in safely when one gets stuck. Use it when you have hands-off tasks running on repeat and want them supervised.

marketing-agent — A marketing strategist and writer that plans campaigns and writes the words for landing pages, emails, social posts, ads, and video scripts. Use it when launching a product or running a marketing campaign.

mle-reviewer — Reviews the code behind machine learning systems (software that learns from data) for reliable data handling, repeatable training, and safe deployment. Use it when changing how a model is trained, served, or monitored.

network-architect — Designs the network for a company or multiple sites based on your requirements, pulling in focused expertise for the routing and validation details. Use it when planning a larger business network.

network-config-reviewer — Checks the settings of network equipment like routers and switches for security mistakes, leftover junk, and risky changes. Use it before applying network configuration changes.

network-troubleshooter — Diagnoses network problems like dropped connections or name-lookup failures using a careful step-by-step, look-but-don't-touch approach, then explains the likely root cause with evidence. Use it when something on the network is broken and you need to find out why.

opensource-forker — Prepares a private project to be shared publicly by copying it, stripping out passwords and secrets, swapping internal references for placeholders, and cleaning its history. Use it as the first step when open-sourcing a project.

opensource-packager — Creates all the standard supporting files a public project needs, like a readme, setup script, license, and contribution guide, so others can use it right away. Use it as a later step when open-sourcing a project.

opensource-sanitizer — Double-checks that a project about to be made public has no leaked passwords, personal information, or sensitive files, and gives a pass or fail report. Use it as a safety check right before any public release.

performance-optimizer — Finds the slow parts of a program and speeds them up, including trimming bloated downloads and fixing memory waste. Use it when something runs too slowly or uses too much

memory.

php-reviewer — An expert reviewer for code written in the PHP language, checking style, security, data handling, and speed. Use it whenever you change PHP code.

planner — A planning expert that breaks down complex features or big changes into a clear, organized roadmap before any building starts. Use it when you're about to take on something complicated and want a solid plan first.

pr-test-analyzer — Reviews the tests that come with a proposed code change to judge whether they truly cover the important behavior and would catch real bugs. Use it when you want to know if a change is properly tested, not just tested on paper.

python-reviewer — An expert reviewer for code written in the Python language, checking style, idioms, security, and performance. Use it whenever you change Python code.

pytorch-build-resolver — Fixes crashes when training AI models with PyTorch (a popular tool for machine learning), like mismatched data shapes and hardware errors, with minimal changes. Use it when PyTorch model training or running fails.

react-build-resolver — Fixes the errors that stop web apps built with React (a popular toolkit for website interfaces) from being assembled, across many different build tools. Use it when a React project won't build.

react-reviewer — An expert reviewer for web interfaces built with React, checking for correct behavior, smooth performance, accessibility, and security. Use it whenever you change React code.

refactor-cleaner — Finds and safely removes code that's no longer used or is duplicated, using tools that detect dead weight. Use it when a project has accumulated clutter you want cleared out.

rust-build-resolver — Fixes the errors that stop programs written in the Rust language from being assembled, including its strict memory rules, with minimal changes. Use it when a Rust project won't compile.

rust-reviewer — An expert reviewer for code written in the Rust language, focused on its memory rules, error handling, and idiomatic style. Use it whenever you change Rust code.

security-reviewer — Hunts for security weaknesses in code, like exposed passwords and common attack openings, and suggests fixes. Use it after writing code that handles logins, user input, or sensitive data.

seo-specialist — Helps your website show up better in search engines like Google by checking your pages, tags, and structure and giving a plan to improve. Use it when you want more people to find your site through search.

silent-failure-hunter — Looks for places where code quietly ignores errors instead of reporting them, which can hide real problems. Use it when you want to make sure failures get noticed rather than swept under the rug.

spec-miner — Reads an existing project and writes out, in a structured form, the rules and behaviors the software is supposed to follow. Use it when adopting a more spec-driven way of working on an older project.

swift-build-resolver — Fixes the errors that stop Apple apps written in the Swift language from being assembled, including signing and dependency issues, with minimal changes. Use it when a Swift project won't build.

swift-reviewer — An expert reviewer for code written in the Swift language used for Apple apps, checking design, memory handling, and idiomatic style. Use it whenever you change Swift code.

tdd-guide — Encourages writing tests before writing the actual feature, so the code is proven to work as you build it. Use it when starting a new feature or fixing a bug and you want strong test coverage from the start.

type-design-analyzer — Reviews how a program defines the shapes of its data to make sure they're well-protected, meaningful, and hard to misuse. Use it when you want feedback on how cleanly your data is modeled.

typescript-reviewer — An expert reviewer for code written in TypeScript and JavaScript, checking safety, correct handling of waiting tasks, security, and good style. Use it whenever you change TypeScript or JavaScript code.

vue-reviewer — An expert reviewer for web interfaces built with Vue (a popular toolkit for website interfaces), checking correct behavior, structure, security, and performance. Use it whenever you change Vue code.

Other global helpers (1)

session-start-hook — A setup helper for the web version of this assistant. It prepares your project so that, when a coding session starts in the browser, the assistant can automatically run the project's tests and quality checks.